

[Kafka] AI 辅助解决 Bug 和性能调优

练习题

习题一 编写数据生成器

数仓的开发流程和进度可以说是由数据驱动的。大多数的数仓搭建分为三个阶段，如采集阶段，分析计算阶段和展示阶段。如果这三个阶段分别由不同的人员来开发。那么下一个阶段必须等到上一个阶段的输出产生才能推动进度。

有一种解决方式是上一个阶段的开发者先约定好自己要产生的最终输出给下游，然后再给出一些模拟数据，最后上游开发完成后，只需替换数据源就可以让下游直接对接，加快数仓的开发搭建。

现在，我们可以使用 GPT 来实现各种数据模拟，现在请写一个数据生成器，生成格式如下的 json 数据 { "user_id": long,

```
"name": string,
```

```
"ts": long,
```

```
"event": string}
```

习题二 获取监控 kafka 的技术方案

虽然监控 Kafka 可以采用多种技术和开源框架，但是当你在网上搜索时，大多数文章都只描述了某一种方案的细节或入门操作。如果你需要了解更多可选的监控方案，那么现在可以向 GPT 提出询问关于 Kafka 监控方案的更多信息。

现在，请你向 GPT 询问 Kafka 的监控方案。

习题三 调整配置以优化 Kafka

问问 GPT 如何优化 Kafka 来提高性能

习题四 解决 Kafka 异常

Kafka 是一个非常稳定的消息队列系统，在大多数情况下，您不需要处理任何异常。然而，本问题旨在提醒您，在遇到异常情况时，您可以随时向 ChatGPT 寻求帮助。

下面是在部署和启动 Kafka 时可能会遇到的错误之一，如果出现此错误，Kafka 将无法启动。

```
org.apache.kafka.common.KafkaException: Failed to acquire lock on file .lock in
/tmp/kafka-logs. A Kafka instance in another process or thread is using this
directory.
```

现在，你可以尝试用 GPT 得到这个异常的解决方案。（这个异常可以通过两次启动 kafka 复现，它会出现在日志目录下的 server.log 文件中）

参考答案

习题一 编写数据生成器

第一轮提示

提示

我想用 java 和 maven 写一个简单的数据生成器，它每 5 秒产生一批数据并将数据写到 Kafka 的 topic 中

我的 Kafka 连接信息是：hadoop102:9092,hadoop103:9092,hadoop104:9092

topic 的名称是 user_test

生成的数据需要时 json 字符串，格式如下

```
{  
  
  "user_id": long,  
  
  "name": string,  
  
  "ts": long,  
  
  "event": string  
  
}
```

生成 10 个 userid

ts 是当前的时间戳

name 和 userid 要一一对应

event 包含 吃 喝 玩 乐四种类型

答

要使用 Java 和 Maven 创建一个简单的数据生成器，首先需要设置一个 Maven 项目。在 `pom.xml` 文件中，添加以下依赖：

```
<project ...>  
  ...  
  <dependencies>  
    <dependency>  
      <groupId>org.apache.kafka</groupId>  
      <artifactId>kafka-clients</artifactId>  
      <version>2.8.1</version>  
    </dependency>  
    <dependency>  
      <groupId>com.fasterxml.jackson.core</groupId>  
      <artifactId>jackson-databind</artifactId>  
      <version>2.12.5</version>
```

```
        </dependency>
    </dependencies>

    ...
</project>
```

接下来，编写一个简单的数据生成器类：

```
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.Producer;
import org.apache.kafka.clients.producer.ProducerRecord;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.io.IOException;
import java.time.LocalDateTime;
import java.time.ZoneId;
import java.util.HashMap;
import java.util.Map;
import java.util.Properties;
import java.util.Random;

public class DataGenerator {

    private static final String[] EVENTS = {"吃", "喝", "玩", "乐"};
    private static final ObjectMapper OBJECT_MAPPER = new ObjectMapper();

    public static void main(String[] args) throws InterruptedException {

        Properties props = new Properties();
        props.put("bootstrap.servers", "hadoop102:9092,hadoop103:9092,hadoop104:9092");
        props.put("key.serializer",
"org.apache.kafka.common.serialization.StringSerializer");
        props.put("value.serializer",
"org.apache.kafka.common.serialization.StringSerializer");

        Producer<String, String> producer = new KafkaProducer<>(props);

        while (true) {
```

```

        for (int i = 1; i <= 10; i++) {
            Map<String, Object> record = new HashMap<>();
            record.put("user_id", i);
            record.put("name", "User_" + i);
            record.put("ts",
LocalDateTime.now().atZone(ZoneId.systemDefault()).toInstant().toEpochMilli());

            record.put("event", getRandomEvent());
            String jsonString = toJsonString(record);

            if (jsonString != null) {
                producer.send(new ProducerRecord<>("user_test", null, jsonString));
                System.out.println("Sent: " + jsonString);
            }
        }
        Thread.sleep(5000);
    }
}

private static String getRandomEvent() {
    Random random = new Random();
    return EVENTS[random.nextInt(EVENTS.length)];
}

private static String toJsonString(Map<String, Object> record) {
    try {
        return OBJECT_MAPPER.writeValueAsString(record);
    } catch (IOException e) {
        e.printStackTrace();
        return null;
    }
}
}

```

这个程序将每 5 秒生成一批数据，并将数据作为 JSON 字符串写入 Kafka 的 `user_test` 主题。`toJsonString` 方法将 `map` 转换为 JSON 字符串。`getRandomEvent` 方法从 `EVENTS` 数组中随机选择一个事件。不要忘了将 Kafka 连接信息的端口更改为适当的值（默认情况下，Kafka 使用 9092 端口）。

反馈

现在，我们可以将代码复制到 IDEA 中进行测试。一旦代码运行起来后，它就会向名为“`user_test`”的 Kafka 主题发送数据。我们可以在控制台中启动一个 `console-consumer` 来消费该主题的数据。

```
[atguigu@hadoop102 module]$ kafka-console-consumer.sh --bootstrap-server hadoop102:9092 --topic user_test
{"user_id":1,"name":"User_1","event":"玩","ts":1681884944083}
{"user_id":2,"name":"User_2","event":"乐","ts":1681884944084}
{"user_id":3,"name":"User_3","event":"吃","ts":1681884944084}
{"user_id":4,"name":"User_4","event":"喝","ts":1681884944084}
{"user_id":5,"name":"User_5","event":"喝","ts":1681884944085}
{"user_id":6,"name":"User_6","event":"喝","ts":1681884944085}
{"user_id":7,"name":"User_7","event":"玩","ts":1681884944085}
{"user_id":8,"name":"User_8","event":"乐","ts":1681884944085}
{"user_id":9,"name":"User_9","event":"玩","ts":1681884944086}
{"user_id":10,"name":"User_10","event":"玩","ts":1681884944086}
{"user_id":1,"name":"User_1","event":"玩","ts":1681884949093}
{"user_id":2,"name":"User_2","event":"乐","ts":1681884949094}
{"user_id":3,"name":"User_3","event":"玩","ts":1681884949095}
{"user_id":4,"name":"User_4","event":"玩","ts":1681884949095}
{"user_id":5,"name":"User_5","event":"玩","ts":1681884949095}
{"user_id":6,"name":"User_6","event":"喝","ts":1681884949095}
{"user_id":7,"name":"User_7","event":"玩","ts":1681884949095}
```

最终，我们会发现该程序能够生成符合我们需求的数据。

如果你想编写更加复杂的数据生成器，还可以继续通过多轮提示来增加需求。

习题二 获取监控 Kafka 的技术方案

第一轮提示

提示

如何监控 kafka 的性能和数据信息。比如有多少消费者消费了某 topic。某个 topic 写入了多少条等等

答

监控 Kafka 性能和数据信息有多种方法，其中包括使用开源工具、查看指标和编写自定义代码。以下是一些建议：

1. 使用开源监控工具：有很多开源工具可以用来监控 Kafka 集群，例如：
 - Kafka Manager (Yahoo CMAK)
 - Confluent Control Center
 - Prometheus + Grafana
 - Kafdrop
 - Burrow
2. 查看 Kafka JMX 指标：Kafka 自带了一套 JMX (Java Management Extensions) 指标，可以提供很多关于集群性能和数据的信息。你可以通过 jconsole 或者其他 JMX 工具查看这些指标。部分关键指标如下：
 - 消费者消费 topic 的速率：

```
kafka.consumer:type=ConsumerTopicMetrics,clientId=<client-id>,topic=<topic-name> -> RecordsPerSec
```
 - topic 写入的速率：

```
kafka.server:type=BrokerTopicMetrics,topic=<topic-name> -> TotalProduceRequestsPerSec
```
 - topic 的数据量：

```
kafka.log:type=Log,name=Size,topic=<topic-name>,partition=<partition-id> -> Value
```
3. 编写自定义代码：你可以编写自定义代码来监控 Kafka 性能和数据信息。Kafka 消费者 API 提供了一些有用的方法，例如：你可以结合这些方法编写代码，以便在你的业务场景中获取关于 Kafka 的实时性能和数据信息。
 - **listTopics()**：列出所有可用主题

- `partitionsFor(topic)`: 获取某个主题的所有分区
- `committed(partition)`: 查询某个消费者在特定分区的消费位置

通过这些方法，你可以监控 Kafka 集群的性能和数据信息，了解消费者消费情况以及 Topic 的写入量等关键数据。这些数据对于调优 Kafka 集群和确保高性能至关重要。

反馈

你可能已经注意到，GPT 为我们提供了许多可选的方案。在关于可选开源框架的部分，GPT 列出了一个简要的列表，不过你可以进一步询问以让 GPT 提供更多关于这些开源框架的说明。

第二轮提示

提示

上面的开源工具中，哪个更适合用来监控 kafka

答

在上面列举的开源工具中，每个工具都有其优点和特点。选择最适合你需求的工具取决于你对监控功能的需求和你的技术栈。以下是一些简要的评估：

1. **Kafka Manager (Yahoo CMAK)**: Kafka Manager 是一个功能齐全的 Kafka 管理和监控工具，提供了 Web 界面，支持主题和消费者管理、分区重新分配、偏移量查看等功能。但由于项目已经有一段时间没有更新，部分功能可能不兼容最新的 Kafka 版本。
2. **Confluent Control Center**: Confluent Control Center 是 Confluent 公司推出的企业级 Kafka 管理和监控工具，提供了丰富的功能和易用的界面。它适用于对性能、稳定性和安全性有更高要求的企业环境。但是，它需要订阅 Confluent 的付费服务，可能不适合预算有限的项目。
3. **Prometheus + Grafana**: Prometheus 和 Grafana 是两个独立的项目，但它们经常一起使用。Prometheus 用于收集和存储指标数据，而 Grafana 用于可视化这些数据。

据。Prometheus 可以通过 JMX Exporter 获取 Kafka 的性能指标。这种组合提供了高度可定制的监控方案，但需要额外的配置和维护工作。

4. **Kafdrop:** Kafdrop 是一个轻量级的 Kafka 管理和监控工具，提供了直观的 Web 界面。它支持查看主题、分区、副本状态以及消费者组的偏移量等基本信息。Kafdrop 易于部署和使用，适合简单的监控需求。

5. **Burrow:** Burrow 是一个 Kafka 消费者健康监控和跟踪工具，主要关注消费者延迟和消费速率等指标。它不提供 Web 界面，而是提供了 REST API。如果你只关心消费者的性能，Burrow 可以是一个不错的选择。

根据你的需求和预算，可以选择最适合你的工具。对于大多数场景，Kafdrop 和 Prometheus + Grafana 可能是较好的选择。Kafdrop 易于使用，适合快速查看基本信息；而 Prometheus + Grafana 则提供了高度可定制的监控方案，适用于需要深入分析性能指标的场景。

反馈

从这两轮问答中可以看出，在进行技术选型时，我们能够更快速地获得一些大体信息，这些信息可以为我们做出技术选型提供方向上的指引。

习题三 调整配置以优化 Kafka

第一轮提示

提示

我 kafka 的 `server.properties` 的配置如下

```
broker.id=2
```

```
num.network.threads=3
```

```
num.io.threads=8
```

```
socket.send.buffer.bytes=102400

socket.receive.buffer.bytes=102400

socket.request.max.bytes=104857600

log.dirs=/opt/data/kafka

num.partitions=1

num.recovery.threads.per.data.dir=1

offsets.topic.replication.factor=1

transaction.state.log.replication.factor=1

transaction.state.log.min.isr=1

log.retention.hours=168 log.segment.bytes=1073741824

log.retention.check.interval.ms=300000

zookeeper.connect=hadoop102:2181,hadoop103:2181,hadoop104:2181/kafka

zookeeper.connection.timeout.ms=6000

group.initial.rebalance.delay.ms=0
```

kafka 集群目前有 3 个节点 hadoop102 hadoop103 hadoop104 里面的 topic 都是三个分区，有没有什么可优化的配置来提高 kafka 的性能

答

根据您提供的 Kafka server.properties 配置信息，这里有一些建议可以帮助您优化 Kafka 集群性能：

1. 增加分区数量： 您目前的 num.partitions 设置为 1，这意味着每个主题只有一个分区。增加分区数量可以提高 Kafka 的并行度，从而提高整体性能。您可以根

据生产和消费的速率以及硬件资源来调整分区数量。将 `num.partitions` 设置为一个更大的值（例如 3、6 或 12）可能会有所帮助。

2. 增加副本数量： 根据您的配置，`offsets.topic.replication.factor` 和 `transaction.state.log.replication.factor` 都设置为 1。这意味着 Kafka 集群只有一个副本来保存数据。为了提高数据的可用性和容错能力，建议您将这两个参数设置为大于 1 的值，例如 2 或 3。请注意，增加副本数量可能会对性能产生一定影响，因此请根据实际情况进行权衡。
3. 调整网络线程和 I/O 线程数量： 根据您的硬件资源，您可以尝试调整 `num.network.threads` 和 `num.io.threads` 的值。如果您的系统具有更多的 CPU 核心，您可以考虑增加这两个参数的值。通常，`num.network.threads` 的值可以设置为 CPU 核心数的 1 到 2 倍，而 `num.io.threads` 的值可以设置为 CPU 核心数的 2 到 4 倍。
4. 调整日志保留策略： 您可以根据业务需求和硬盘空间调整日志保留策略。例如，可以根据实际情况增加或减少 `log.retention.hours` 的值。此外，还可以考虑调整 `log.retention.bytes` 参数，以限制日志保留的总字节数。
5. 调整 socket 参数： 您可以根据您的网络环境和系统资源来调整 `socket.send.buffer.bytes` 和 `socket.receive.buffer.bytes` 的值。这两个参数分别用于设置发送和接收缓冲区的大小。通常情况下，这些值可以设置为 32KB、64KB 或 128KB。如果您的网络具有更高的带宽和更低的延迟，可以尝试增加这些值以提高性能。
6. 启用压缩： 为了减少网络传输和磁盘存储的开销，您可以考虑启用 Kafka 的压缩功能。Kafka 支持多种压缩算法，如 GZIP、Snappy 和 LZ4。启用压缩功能需要在生产者和消费者的配置中进行设置。例如，可以将 `producer.properties` 中的 `compression.type` 参数设置为 "gzip"、"snappy" 或 "lz4"。

请注意，这

反馈

内容没有写完，此时可以恢复“继续”让 GPT 继续生成内容

第二轮提示

提示

继续写

答

些优化建议可能会根据您的实际使用场景和硬件资源产生不同的效果。在进行任何更改之前，请务必对当前的 Kafka 集群进行性能基准测试。在应用优化建议后，也要进行性能测试，以确保这些更改实际上提高了 Kafka 集群的性能。以下是一些额外的优化建议：

1. 批量发送： 可以通过调整 `producer.properties` 中的 `batch.size` 和 `linger.ms` 参数来实现消息的批量发送。这将允许生产者将多个消息一起发送，从而提高吞吐量。`batch.size` 参数表示一个批次中允许的最大消息字节数，而 `linger.ms` 参数表示生产者等待消息进入批次的最长时间（以毫秒为单位）。适当调整这两个参数可以提高 Kafka 的性能。
2. 调整消费者拉取消息的频率和批次大小： 消费者可以通过调整 `fetch.min.bytes` 和 `fetch.max.wait.ms` 参数来控制拉取消息的频率和批次大小。`fetch.min.bytes` 参数表示消费者从 Kafka 拉取的最小字节数，而 `fetch.max.wait.ms` 参数表示消费者等待达到最小字节数的最长时间（以毫秒为单位）。根据您的实际需求和系统资源调整这些参数，可以帮助提高消费者的性能。
3. 调整 Java 虚拟机（JVM）设置： Kafka 运行在 JVM 上，因此调整 JVM 参数也可能对性能产生影响。您可以通过修改 `kafka-server-start.sh` 脚本中的 `KAFKA_HEAP_OPTS` 和 `KAFKA_JVM_PERFORMANCE_OPTS` 参数来调整 JVM 设置。例

如，您可以增加初始堆大小（-Xms）和最大堆大小（-Xmx）以提高 Kafka 的性能。同时，请确保您的系统具有足够的内存资源。

4. 监控和调优操作系统： 为了充分利用 Kafka 的性能，建议您密切关注操作系统的性能指标，例如 CPU 利用率、内存使用情况、磁盘 I/O 和网络吞吐量。您可以使用诸如 `iostat`、`vmstat` 和 `netstat` 等工具来监控这些指标，并根据实际情况进行相应的调优。

请注意，优化 Kafka 集群性能是一个持续的过程。在应用任何优化建议后，请确保持续监控 Kafka 集群的性能，以便根据需求进行相应的调整。

反馈

这些优化建议都是非常准确的。这些优化建议非常准确。如果你提供了更多的信息，例如网卡型号、磁盘型号、数据量等，它就可以为您提供更具实施价值的建议。

习题四 解决 Kafka 异常

第一轮提示

提示

我的 Kafka 启动时遇到如下异常 `org.apache.kafka.common.KafkaException: Failed to acquire lock on file .lock in /tmp/kafka-logs. A Kafka instance in another process or thread is using this directory.` 该如何处理。

答

这个异常表明 Kafka 无法在其日志目录（这里是 `/tmp/kafka-logs`）上获取文件锁，可能是因为另一个 Kafka 实例或进程已经在使用这个目录。要解决这个问题，你可以尝试以下方法：

1. 确认是否有其他 Kafka 实例正在运行：使用 `ps aux | grep kafka` 命令来检查是否有其他 Kafka 实例正在运行。如果发现有其他实例，请停止它们。

2. 删除.lock文件：如果你确定没有其他 Kafka 实例正在运行，可以尝试删除.lock文件。在本例中，执行 `rm /tmp/kafka-logs/.lock`。但请注意，在删除.lock文件之前，请确保确实没有其他 Kafka 实例正在运行，以免造成数据损坏。
3. 更改 Kafka 日志目录：如果问题仍然存在，可以尝试更改 Kafka 的日志目录。在 Kafka 配置文件（`server.properties`）中，找到 `log.dirs` 设置并更改为其他目录，例如：`log.dirs=/tmp/kafka-logs-1`。然后重启 Kafka。
4. 确认权限：确保 Kafka 进程具有足够的权限访问和操作日志目录。如果权限不足，可以使用 `chmod` 和 `chown` 命令调整目录权限和所有权。

尝试上述方法后，应该可以解决 Kafka 启动时遇到的异常。如果问题仍然存在，请检查 Kafka 的日志以了解更详细的错误信息，并根据具体情况进行排查。

反馈

这种异常实际上是在同一台机器上启动了多个 Kafka 实例时可能遇到的问题。当后启动的 Kafka 实例检测到具有相同日志目录的先前实例已经在运行时，它会尝试在该目录下创建一个建议性记录锁，以防止并发访问。如果该目录下已经存在了一个.lock 锁标记，则 Kafka 将无法启动，并抛出本题中所描述的异常。

因此，GPT 给出的解决思路是正确的。